

Laboratoire 7

Driver de Linux

Départements : TIC
Unité d'enseignement SCF

Auteur : Urs Behrmann
Professeurs : Alberto Dassatti
Yann Thomas
Assistant : Clément Dieperink
Classe : SCF
Salle de labo : A09
Date : 28.04.2026

Contents

1	Objectif	3
2	Interface mise en place	3
3	Application user-space	4
4	Validation	4
5	Utilisation d'outil IA	5

1 Objectif

L'objectif du laboratoire est de développer un driver Linux permettant d'interfacer notre IP AXI4-Lite avec une application user-space. Le driver doit offrir une interface simple d'utilisation depuis Linux, tout en vérifiant à l'initialisation que le design chargé dans le FPGA est correct.

2 Interface mise en place

L'interface choisie est un **driver caractère** exposé via le noeud `/dev/axi_lw`. Le module est chargé avec `insmod`, puis l'application user-space ouvre ce device et communique avec le kernel par `ioctl`.

Le driver utilise un `miscdevice` afin de réduire le code de création du périphérique (pas de `cdev/class/device` manuel).

Justification du choix

J'ai choisi un driver caractère chargé manuellement, en cohérence avec ce qui a été vu dans le cours de drivers Linux et pour garder une mise en oeuvre simple dans le cadre du laboratoire. Une alternative aurait été un driver auto-chargé (*platform driver*), mais cette approche demandait d'adapter le Device Tree. Pour ce laboratoire, l'effort supplémentaire côté Device Tree n'était pas nécessaire. Le compromis retenu est donc d'utiliser `insmod` à chaque itération de test, ce qui restait acceptable par rapport à la complexité d'une adaptation DT.

Accès matériel

Le module mappe la zone AXI lightweight à l'adresse `0xFF200000` (`ioremap`). Au chargement, il lit le registre constant (`REG_CST`) et compare avec `0xBADB100D`. Si la valeur ne correspond pas, le module retourne `-ENODEV` et refuse de s'initialiser.

API ioctl utilisée

Les commandes `ioctl` retenues pour l'application de test sont:

- lecture: `GET_CST`, `GET_TEST`, `GET_SWITCH`, `GET_KEYS_EDGE`
- écriture: `SET_TEST`, `SET_LEDS`, `CLR_LEDS`, `SET_HEX3_0`, `SET_HEX5_4`

Le choix de `GET_KEYS_EDGE` (avec acquittement automatique) permet un comportement événementiel: une action par appui de touche, sans répétition continue pendant un appui long.

3 Application user-space

L'application `axi_ctl.c` a été adaptée pour reprendre la logique du labo 6:

- lecture et vérification de la constante
- écriture/lecture du registre de test
- boucle principale avec gestion des touches `KEY0..KEY3`
- affichage de la valeur sur les 7 segments et gestion des LEDs

La différence principale est que l'accès matériel direct (`/dev/mem`) est remplacé par l'interface du driver.

4 Validation

Les tests effectués:

- compilation du module (`axi_driver.ko`)
- chargement avec `insmod` et création de `/dev/axi_lw`
- exécution de l'application user-space et vérification du comportement

Le module se charge correctement et l'application permet de piloter les registres de l'IP via `ioctl`.

5 Utilisation d'outil IA

Pour ce rapport, j'ai utilisé un assistant IA pour m'aider à structurer les étapes et à reformuler certaines phrases. Les manipulations techniques, les tests et les validations ont été réalisés par moi.